Documentation of the HIF–Iron Homeostasis Simulation Code

Irina Piazza
with the original framework by Eric Fanchon and Alexandre Rocca

TIMC-IMAG, CNRS, Grenoble, France

Abstract

This report documents the Python simulation framework used to explore the parameter space of a cellular iron homeostasis model and its extension toward HIF–iron coupling. It describes the software architecture of the public release, how to install and run the code, the sampling and filtering workflow, and selected technical notes (including stiffness diagnostics). Historical laboratory protocols are retained in corrected English for reproducibility of earlier campaigns; command names are updated to the current public API (`simulate.py`, `config.py`, `filter.py`, `model.py`).

Contents

1 Overview **2**
- 1.1 Scientific context . . . 2
- 1.2 Software lineage . . . 2
- 1.3 Architecture of the public code . . . 2
- 1.4 Typical pipeline . . . 3

2 Installation and usage **4**
- 2.1 Environment setup . . . 4
- 2.2 Running a simulation . . . 4
- 2.3 Outputs . . . 5
- 2.4 Regression tests . . . 5

3 Historical sampling protocol **6**
- 3.1 Integrated simulate + filter workflow . . . 6
- 3.2 Outline of the refinement campaign . . . 6
- 3.3 Example commands (updated names) . . . 7
- 3.4 Comparison with A. Rocca's box . . . 7
- 3.5 Notes on interpretation . . . 7

4 Stiffness investigation **8**
- 4.1 Definition used in this work . . . 8
- 4.2 Implementation in the code . . . 8
- 4.3 Practical remarks . . . 8

Chapter 1

Overview

1.1 Scientific context

Iron homeostasis is tightly regulated at the cellular level. The models studied in this project describe key species (labile iron, ferritin and iron storage, IRP1, TfR1, ferroportin, and later HIF) and their kinetic interactions under different iron-replenishment conditions. The numerical goal is to sample a high-dimensional parameter space, retain only trajectories that satisfy biological constraints, and characterise the resulting admissible parameter regions.

1.2 Software lineage

The original simulation framework was developed by Eric Fanchon and Alexandre Rocca (TIMC-IMAG, CNRS). Irina Piazza later reorganised the code for clarity and maintainability: configuration was centralised, input/output was standardised with pandas DataFrames, filtering and logging were rewritten, plotting utilities were added, and the ODE right-hand side was unified. The public release distributed with this documentation is the HIF–iron oriented version of that work, modernised for Python 3.

Changes introduced by Irina Piazza are marked in the source with comments of the form `# IP <date>`.

1.3 Architecture of the public code

The main modules are:

`simulate.py` Entry point. Implements sampling modes (`Box-Sampling`, `Re-Sampling`, `Shell-Sampling`, `PCA-guided-sampling`, ...), numerical integration, constraint checks, optional filtering, dumping of trajectories, and plotting.

`model.py` Model-specific ODE system, algebraic steady-state construction, absolute/relative constraints, and optional stiffness diagnostics.

`config.py` Global constants: solver tolerances, sampling settings, variable/parameter names, bounds and biological constraints.

`filter.py` Post-processing filter on ratios and concentration bounds; writes filtered solution lists and enclosing boxes.

`read_file.py` Reads bound files and results tables.

`log.py` Writes `info.txt` and copies input files into each run directory.

`sobol_lib.py`, `pca.py` Quasi-Monte Carlo sampling and PCA-guided resampling helpers.

`plot_*.py`, `multiplot_*.py` Offline visualisation utilities.

Each execution creates a timestamped folder `Run_DD_MM_YYYY-HhMmSs/` containing results, plots, filtered outputs (when applicable), and run metadata.

1.4 Typical pipeline

1. Sample parameter vectors inside a box (Sobol sequence) or around previous solutions (resampling / shell / PCA modes).
2. For each vector, compute algebraic initial conditions, improve the steady state numerically if needed, then integrate the ODEs for each iron input listed in `MULTIPLE_INPUTS`.
3. Accept the set only if absolute and relative constraints are satisfied.
4. After the sampling loop, filter the accepted set with `filter.py`, dump a subset of trajectories, and produce plots.

Chapter 2

Installation and usage

2.1 Environment setup

Create an isolated Python environment and install dependencies from the project root:

```
python -m venv .venv
# Linux/macOS:
source .venv/bin/activate
# Windows PowerShell:
# .\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip
pip install -r requirements.txt
python tests/run_all_tests.py
```

Required packages include NumPy, SciPy, pandas, matplotlib and recordclass (see `requirements.txt`).

2.2 Running a simulation

Basic command:

```
python simulate.py b-vars-hif.txt b-par-hif.txt results.txt Box-Sampling 1000
```

Arguments:

1. Variable bounds file (`name lower_bounds upper_bounds`).
2. Parameter bounds file, or a previous results / filter-solution file depending on the mode.
3. Output results file name (written under the new `Run_*` folder).
4. Mode (optional; default is box sampling when the CLI provides five arguments as above).
5. Number of sampled points / iterations.

Useful modes:

`Box-Sampling` Sample the parameter box with a Sobol sequence.

`Re-Sampling` Build local boxes around previously accepted solutions and sample them.

`Shell-Sampling` Sample a shell of controlled width around a previous solution set (`magnitude` in `config.py`).

`PCA-guided-sampling` Resample using a PCA reduction of the accepted cloud.

Key settings live in `config.py`: solver tolerances (`SIMU_RTOL`, `SIMU_ATOL`), steady-state criteria, iron inputs (`MULTIPLE_INPUTS`), biological bounds, and filter ratios.

2.3 Outputs

A typical run directory contains:

- the results table (parameters, initial variables, ratios, total iron);
- `Solutions_y_t/` and `Solutions_filtered_y_t/` trajectory dumps (a capped subset, to keep files manageable);
- plot folders for raw and filtered solutions, and parameter correlations;
- `Filter_sol_*.txt` / `Filter_box_*.txt` when filtering succeeds;
- `info.txt` with settings and copied input bound files.

2.4 Regression tests

Contract and numerical fingerprint tests protect against unintentional changes:

```
python tests/run_all_tests.py
```

Golden cases are stored under `tests/golden/`. Regenerate them only after a deliberate, reviewed numerical change:

```
python tests/generate_golden.py
```

Chapter 3

Historical sampling protocol

This chapter summarises laboratory campaigns performed while refining the iron-homeostasis parameter region (late 2019 – early 2020). The narrative is preserved for reproducibility of those results. In the present public code, the entry point is `simulate.py` (formerly `simulate_test.py` / `simulate_P.py`), configuration lives in `config.py`, and filtering is integrated after sampling (formerly a separate `filter_test.py` call in early workflows).

3.1 Integrated simulate + filter workflow

In later versions of the code, filtering is invoked in the same run as sampling: once `apply_method()` returns candidate solutions, `filter()` retains only those that satisfy the ratio and concentration constraints. The code also writes trajectory files and plots for each variable and each iron experiment. With seven variables and four iron inputs this yields up to 28 trajectory files and 28 plots for both the unfiltered and filtered sets.

Because the number of accepted solutions can be large, trajectory dumping and plotting are capped (on the order of 100 trajectories written and a smaller number plotted) so that output remains manageable. The main results table stores parameters, initial variables, ratios (Fe-sto, TfR1, Ft) and total iron for each experiment in a DataFrame-like columnar layout.

3.2 Outline of the refinement campaign

A representative sequence was:

1. **Box sampling** of a large initial box (millions of Sobol points) with relatively relaxed ferritin-ratio bounds at 250 nM iron replenishment (e.g. Ft ratio in $[4, 12]$).
2. **Filtering**, which may initially retain very few or no points.
3. Repeated **re-sampling** around the surviving solutions, alternating with filtering, to densify the admissible region.
4. Tightening of the ferritin ratio at 250 nM to a validated interval such as $[4, 6.5]$.
5. Optional **shell sampling** to explore the boundary of the current box, followed again by re-sampling / box sampling to restore a more uniform coverage.
6. Comparison of the enclosing box volume with the reference box obtained by A. Rocca, and occasional enlargement of weakly constrained directions (e.g. $k_{\mathrm{IRP_FPN1a}}$) when biological arguments suggested a wider admissible range.

Table 3.1 gives a compact schematic of one such campaign (counts are those recorded at the time; exact folder names varied between runs).

Step	Mode	Iterations	Ft ratio at 250 nM	Filtered sols.
1	Box-Sampling	4×10^6	$[4.0, 12.0]$	0 after filter
2A	Re-Sampling (0.2)	10^3	$[4.0, 12.0]$	1
2B	Re-Sampling (0.2)	10^5	$[4.0, 12.0]$	12421
2C	Re-Sampling	(dense)	$[4.0, 6.5]$	~ 922
later	Shell / Box / Re-Sampling	up to 5×10^6	$[4.0, 6.5]$	variable

Table 3.1: Schematic of the historical refinement steps. Intermediate unfiltered counts were typically much larger than the filtered counts.

3.3 Example commands (updated names)

Initial box sampling:

```
python simulate.py b-vars-hif.txt b-par-hif.txt results.txt Box-Sampling 4000000
```

Re-sampling around previous solutions (use a results or filter-solution file as the second argument):

```
python simulate.py b-vars-hif.txt previous_Filter_sol.txt results_rs.txt Re-Sampling
    100000
```

Shell sampling:

```
python simulate.py b-vars-hif.txt previous_Filter_sol.txt results_shell.txt Shell-
    Sampling 4000000
```

3.4 Comparison with A. Rocca's box

At intermediate stages the enclosing volume of the newly obtained box was compared with A. Rocca's reference volume. When the new box became much smaller, shell sampling and controlled enlargement of selected parameter ranges were used to avoid an artificially contracted region. Merged boxes combining both analyses were also sampled in box-sampling mode to assess the joint admissible set.

3.5 Notes on interpretation

- Filtered counts depend strongly on the ratio bounds in `config.py`; relaxing or tightening those bounds changes the accepted set dramatically.
- Re-sampling densifies local neighbourhoods but can bias coverage; a subsequent box-sampling pass on the enclosing box restores a more uniform distribution.
- Parameter-correlation scatter plots (see `plot_params_correlation.py` and the `multiplot_*` scripts) are useful to visualise possible dependencies in the accepted cloud.

Chapter 4

Stiffness investigation

4.1 Definition used in this work

A linear constant-coefficient system

$$\mathbf{y}' = A\mathbf{y} + \mathbf{f}(x)$$

is considered stiff when all eigenvalues of A have negative real parts and the stiffness ratio

$$\frac{|\mathrm{Re}(\overline{\lambda})|}{|\mathrm{Re}(\underline{\lambda})|}$$

is large, where $\overline{\lambda}$ and $\underline{\lambda}$ are the eigenvalues with the largest and smallest $|\mathrm{Re}(\lambda)|$, respectively. Fast transients then coexist with slow ones, which is numerically demanding for explicit integrators.

Although the iron-homeostasis ODEs are nonlinear, the Jacobian can be evaluated at a given time point. Locally, the system may then be treated as linear for the purpose of estimating a stiffness ratio.

4.2 Implementation in the code

In `model.py`, inside `diff_eqs()`, the Jacobian is assembled in the time window corresponding to the iron perturbation (approximately $t \in [10000, 30000]$ s, i.e. about 2.7–8.3 h). Eigenvalues are computed with `scipy.linalg.eig`. When negative real parts are found and pairwise stiffness ratios exceed a threshold (e.g. 100), a short report is written to a stiffness log file.

This diagnostic is optional and intended for model analysis; it does not change the acceptance criteria of the sampling pipeline.

4.3 Practical remarks

- Stiffness, when present, motivates the use of implicit or stiff-capable integrators (the default path uses SciPy's `odeint` / LSODA-style switching).
- The diagnostic depends on the current parameter set and on the state trajectory; a system may look stiff for some accepted parameters and not for others.
- Extending the model with HIF-related reactions increases the state dimension and can alter the eigenvalue spectrum; the same Jacobian-based check remains applicable after updating the matrix assembly.